

Modern İşletim Sistemleri

BÖLÜM 1: GİRİŞ

Andrew S. Tanenbaum & Herbert Bos — 5. Baskı

İÇİNDEKİLER

- 1.1 İşletim Sistemi Nedir?
- 1.2 İşletim Sistemlerinin Tarihi
- 1.3 Bilgisayar Donanımı Gözden Geçirme
- 1.4 İşletim Sistemi Hayvanat Bahçesi
- 1.5 İşletim Sistemi Kavramları
- 1.6 Sistem Çağrılar (System Calls)
- 1.7 İşletim Sistemi Yapısı
- Quiz Soruları ve Cevapları
- Bölüm Sonu Soruları ve Çözümleri

Tüm açıklamalar Türkçe · İngilizce teknik terimler korunmuştur

1.1 İşletim Sistemi Nedir? (What is an Operating System)

Modern bir bilgisayar; işlemciler, ana bellek, sabit disk ya da Flash sürücüler, klavye, fare, ekran ve ağ arayüzü gibi pek çok bileşenden oluşur. Bu kadar karmaşık bir sistemi doğrudan yönetmek son derece güçtür. İşletim sistemi (operating system), uygulama programlarına bilgisayar donanımının güzel ve sade bir soyutlamasını sunmak ve tüm donanım kaynaklarını yönetmek üzere bu karmaşıklığın üzerine eklenen yazılım katmanıdır.

Çift Rol

İşletim sistemi iki temel işlev yerine getirir: (1) Uygulama programlarına temiz ve tutarlı soyutlamalar sunmak (üst-aşağı bakış), (2) Donanım kaynaklarını verimli ve hakkaniyetli biçimde yönetmek (aşağı-yukarı bakış).

Yazılım Katmanları (Software Layers)

Bir bilgisayar sistemi genel olarak üç katmandan oluşur:

Katman	Çalışma Modu	Örnekler	Açıklama
Kullanıcı Uygulamaları	Kullanıcı Modu (User Mode)	Tarayıcı, e-posta, medya oynatıcı	Sınırlı donanım erişimi
Kullanıcı Arayüzü (Shell/GUI)	Kullanıcı Modu	Bash, Zsh, Windows Desktop, GNOME	Kullanıcı ile OS arası köprü
İşletim Sistemi (OS)	Çekirdek Modu (Kernel Mode)	Linux Kernel, Windows NT Kernel	Donanıma tam erişim
Donanım (Hardware)	—	CPU, RAM, Disk, GPU, NIC	Fiziksel bileşenler

Kernel Mode vs User Mode

Kernel mode (çekirdek modu): İşlemcinin tüm komutlarını çalıştırabildiği ve tüm donanım kaynaklarına erişebildiği özel mod. User mode (kullanıcı modu): Yalnızca belirli komutların çalıştırılabildiği, I/O ve bellek koruma komutlarının yasak olduğu kısıtlı mod. İki mod arasındaki geçiş System Call (sistem çağırısı) mekanizmasıyla gerçekleşir.

1.1.1 Genişletilmiş Makine Olarak İşletim Sistemi (Extended Machine)

Çoğu bilgisayarın makine dili düzeyindeki mimarisi son derece ilkel ve programlaması zordur. Örneğin SATA hard disk arayüzünü doğrudan kullanan bir kitap 450 sayfayı aşmaktadır. Bir disk sürücüsü (disk driver), bu karmaşıklığı gizleyerek programcıya yalnızca 'disk bloğu oku/yaz' gibi basit bir arayüz sunar. İşletim sistemi üst üste bindirilmiş soyutlama (abstraction) katmanlarıyla donanımın çirkin yüzünü güzel bir arayüze dönüştürür.

Soyutlama Seviyesi	Uygulama Programcısının Gördüğü
Ham Donanım (SATA komutları)	450+ sayfalık teknik kılavuz
Disk Sürücüsü (Disk Driver)	read_block(), write_block() API
Dosya Sistemi (File System)	open(), read(), write(), close()
Uygulama Programı	Dosya adıyla basit okuma/yazma

1.1.2 Kaynak Yöneticisi Olarak İşletim Sistemi (Resource Manager)

Aşağı-yukarı bakışa göre işletim sistemi karmaşık bir sistemin tüm parçalarını yönetir. CPU, bellek ve I/O aygıtlarının birden fazla program arasında düzenli ve kontrollü biçimde paylaşılmasını sağlar. Kaynak paylaşımı (resource multiplexing) iki farklı biçimde gerçekleşir:

Paylaşım Türü	Tanım	Örnek Kaynaklar
Zamanda Çoklama (Time Multiplexing)	Kaynağı programlar sırayla kullanır	CPU, Yazıcı
Uzayda Çoklama (Space Multiplexing)	Her program kaynağın bir parçasını alır	RAM, Disk/SSD

⚠ Yazıcı Kaos Örneği

Yazıcı örneği: 3 program aynı anda yazdırmak isterse çıktı birbirine karışır. OS, çıktıları önce disk/SSD'ye tamponlar (spooling), program bitince sırayla yazdırır — bu klasik zaman çoklama yönetimidir.

1.2 İşletim Sistemlerinin Tarihi (History of Operating Systems)

İşletim sistemleri bilgisayar mimarisinin evrimi ile birlikte beş nesil boyunca gelişmiştir. Her nesil, öncekinin eksikliklerini gidermek için ortaya çıkmıştır.

Nesil	Dönem	Teknoloji	OS Özellikleri	Temel Kavramlar
1. Nesil	1945–1955	Vakum tüpleri (Vacuum Tubes)	İşletim sistemi yok; makine doğrudan kullanılır	Plug-board programlama; her iş için yeniden bağlama
2. Nesil	1955–1965	Transistörler; Batch Systems	Basit batch monitörleri; iş kartı okuyucular	Spooling; FMS (Fortran Monitor System); IBSYS
3. Nesil	1965–1980	Entegre devreler (IC); Müşterek bellek	Multiprogramming; Timesharing (CTSS, MULTICS)	OS/360; UNIX (1969); POSIX standardı
4. Nesil	1980–Günü müz	LSI çipleri; Kişisel bilgisayar	CP/M → MS-DOS → Windows; macOS; Linux	GUI; ağ OS; dağıtık OS; SMP desteği
5. Nesil	1990–Günü müz	Mobil çipler; Dokunmatik ekran	iOS; Android (Linux tabanlı); Wearable OS	ARM mimarisi; güç tasarrufu; bulut entegrasyonu

Önemli Tarihsel Kilometre Taşları

Yıl	Olay
1956	İlk disk sürücüsü: IBM RAMAC 350 (5 MB, buzdolabı büyüklüğünde)
1964	IBM OS/360: ilk genel amaçlı büyük OS (milyonlarca satır assembly)
1969	Ken Thompson Bell Labs'da UNIX'i yazar (PDP-7 üzerinde)
1974	Intel 8080: ilk genel amaçlı 8-bit CPU
1977	CP/M: mikrocomputer pazarına hakim OS
1981	MS-DOS: IBM PC ile birlikte piyasaya çıkar
1984	Apple Macintosh: kitlesel GUI devrimini başlatır
1987	Tanenbaum, MINIX'i yayımlar (eğitim amaçlı)
1991	Linus Torvalds, Linux'u yazar (MINIX'ten ilham alarak)
2007	Apple iOS: modern mobil OS çağını açar
2008	Android 1.0: Linux tabanlı, açık kaynak mobil OS

■ MULTICS Mirası

MULTICS (1965–2000): MIT, Bell Labs ve GE'nin birlikte geliştirdiği devrimci sistem. Sanal bellek, hiyerarşik dosya sistemi ve segmentasyon gibi pek çok kavramı ilk kez hayata geçirdi. UNIX ve dolayısıyla Linux/macOS/iOS/Android'in atasıdır.

1.3 Bilgisayar Donanımı Gözden Geçirme (Computer

İşletim sisteminin doğru anlaşılması için altında çalıştığı donanımın iyi bilinmesi gerekir. Bu bölüm CPU, bellek hiyerarşisi, I/O aygıtları, bus mimarisi ve bilgisayarın açılış (boot) sürecini kapsamaktadır.

1.3.1 İşlemciler (Processors / CPU)

CPU'nun temel döngüsü 'fetch-decode-execute' (getir-çözümle-çalıştır) döngüsüdür. Her döngüde programı sayacı (PC/IP - Program Counter/Instruction Pointer) tarafından gösterilen bellekteki komut okunur, çözümlenir ve çalıştırılır.

Register (Yazmaç)	Kısaltma	Görevi
Program Counter	PC / IP	Sıradaki çalışacak komutun adresini tutar
Stack Pointer	SP	Yığının (stack) tepesini gösterir
Program Status Word	PSW	Mod biti (user/kernel), koşul bitleri, CPU önceliği
Genel Amaçlı Yazmaçlar	r0..rN	Aritmetik ve veri işlemleri için geçici depolama

CPU Mimarisi	Açıklama	Avantaj
Pipeline	Komutları üst üste binen aşamalarda işler (fetch → decode → execute paralel)	Saat hızından bağımsız verim artışı
Superscalar	Birden fazla execution unit; komutlar sıra dışı çalışabilir	IPC (Instructions Per Cycle) > 1
Multithreading / Hyperthreading	Bir çekirdek birden fazla thread durumunu tutar; nanosaniye aralarla geçiş yapar	I/O bekleme sürelerini gizler
Multicore	Tek çipte birden fazla tam CPU çekirdeği (4, 8, 16... çekirdek)	Gerçek paralel çalışma
GPU	Binlerce küçük çekirdek; SIMD operasyonlar	Grafik & paralel hesaplama

i Moore Yasası

Moore Yasası: Bir çipteki transistör sayısı yaklaşık her 18 ayda bir ikiye katlanır (Gordon Moore, 1965). Bu 'yasa' fizik yasası değil, bir gözlemdir ve atom boyutlarına yaklaşıldıkça kuantum etkileri nedeniyle yavaşlamaktadır.

⚠ Mode Biti Güvenlik Temeli

Kernel mode / User mode farkı: PSW'deki tek bir bit, CPU'nun tüm komutlara erişip erişemeyeceğini belirler. Kullanıcı programı kernel moduna geçmek için SYSTEM CALL yapmak zorundadır — doğrudan PSW değiştiremez.

1.3.2 Bellek Hiyerarşisi (Memory Hierarchy)

İdeal bellek hem son derece hızlı, hem çok kapasiteli, hem de ucuz olurdu; ancak günümüz teknolojisinde bu üç özelliği birden karşılayan bir çözüm yoktur. Bu nedenle bellek hiyerarşik katmanlar halinde tasarlanır: yukarıdaki katmanlar daha hızlı, daha pahalı ve daha küçüktür; aşağıdakiler daha yavaş ama ucuz ve büyüktür.

Katman	Erişim Süresi	Tipik Kapasite	Kontrol Eden	Özellik
CPU Yazmaçları (Registers)	< 1 ns	< 1 KB	Yazılım (programcı)	CPU içinde, en hızlı
L1 Cache	1-2 ns	32-64 KB (çekirdek başına)	Donanım	CPU çipiyle bütünleşik
L2 Cache	3-10 ns	256 KB - 4 MB	Donanım	Bazen paylaşımlı
L3 Cache	10-30 ns	8-64 MB	Donanım	Çekirdekler arası paylaşım
Ana Bellek (RAM)	50-100 ns	16-64 GB	OS + Donanım (MMU)	DRAM; güç gitince silinir
SSD (Flash)	10-100 µs	256 GB - 4 TB	OS + SSD Firmware	Kalıcı; mekanik parça yok
Manyetik Disk (HDD)	5-10 ms	2 - 16+ TB	OS + Disk Denetçisi	Kalıcı; mekanik; çok ucuz/bit

Cache Mantığı

Cache (önbellek) bir kaynağın en yoğun kullanılan parçalarını daha hızlı bir katmanda tutmak için kullanılan temel bir tekniktir. Ana bellekte her 64 byte'lık blok bir cache line'a karşılık gelir.

Durum	Açıklama	Maliyet
Cache Hit (İsabet)	İstenen veri önbellekte; doğrudan servis edilir	~2 ns (çok düşük)
Cache Miss (İskalama)	Veri önbellekte yok; ana bellekten yüklenir	~50-100 ns (yüksek)
Page Fault	Veri RAM'de yok; diskten yüklenir	~5-10 ms (çok yüksek)

Cache Hız Kazanımı Hesabı

Performans örneği: Bir programın %95 oranında cache hit'i varsa ve %5 cache miss durumunda RAM'e gidiyorsa: ortalama erişim süresi = $0.95 \times 2\text{ns} + 0.05 \times 50\text{ns} = 1.9 + 2.5 = 4.4\text{ ns}$. Cache olmadan her erişim 50 ns sürecekti — ~11 kat hız kazanımı!

Bellek türleri özeti:

Bellek Türü	Uçucu mu?	Programlanabilir mi?	Kullanım Amacı
RAM (DRAM)	Evet	Evet (işletim sırasında)	Ana çalışma belleği
ROM	Hayır	Hayır (fabrika ayarı)	Boot kodu (eski sistemler)
EEPROM / Flash	Hayır	Evet (özel protokolle)	BIOS/UEFI firmware; SSD
CMOS	Evet (pil ile korunur)	Evet	Saat, tarih, boot ayarları
Persistent Memory (Intel Optane)	Hayır	Evet	RAM hızında kalıcı depolama

1.3.3 Kalıcı Depolama (Nonvolatile Storage)

Manyetik diskler (HDD), SSD'ler ve kalıcı bellek (persistent memory), elektrik gitse de veri saklar. Diskler mekanik yapıları nedeniyle yavaş ama çok ucuzdur. SSD'ler Flash bellek kullanır; mekanik parça olmadığı için rastgele erişimde çok hızlıdır. Ancak fazla yazma-silme döngüsü yıpratır.

Özellik	Manyetik Disk (HDD)	SSD (Flash)	Persistent Memory
Okuma Süresi	5-10 ms	10-100 µs	~100 ns
Yazma Süresi	5-10 ms	100-1000 µs	~200 ns
Kapasite/Fiyat	En ucuz/bit	Orta	Pahalı
Mekanik Parça	Evet (dönen plaka)	Hayır	Hayır
Aşınma	Mekanik aşınma	Write cycle sınırı	Çok az
Erişim Arayüzü	Blok (512 B/4 KB)	Blok (4 KB)	Byte düzeyinde

1.3.4 Giriş/Çıkış Aygıtları (I/O Devices)

I/O aygıtları genel olarak iki parçadan oluşur: denetçi (controller) ve aygıtın kendisi. Denetçi (aygıt sürücüsünün muhatap olduğu), karmaşık donanım ayrıntılarını gizleyerek OS'e basit bir arayüz sunar.

I/O Yöntemi	Açıklama	CPU Kullanımı	Avantaj / Dezavantaj
Programlı G/Ç (Busy Waiting / Polling)	CPU sürekli aygıtın bitmesini yoklar	Çok yüksek (%100)	Basit ama CPU israfı büyük
Kesme Tabanlı G/Ç (Interrupt-Driven I/O)	Aygıt bitince CPU'ya kesme (interrupt) gönderir	Düşük (interrupt süresi kadar)	Etkin; küçük transferler için ideal
Doğrudan Bellek Erişimi (DMA - Direct Memory Access)	DMA çipi bellek↔aygıt transferini CPU'suz yapar; tamamlanınca interrupt	En düşük (sadece init + interrupt)	Büyük blok transferleri için ideal

i BİLGİ

Interrupt vektör tablosu (interrupt vector table): Her aygıtın interrupt işleyicisinin (handler) bellek adresini tutan tablo. CPU bir interrupt aldığı anda PSW ve PC'yi yığına iter, kernel moduna geçer, vektör tablosundan ilgili handler adresini okur ve handler'ı çalıştırır. Handler bitince kesilen program kaldığı yerden devam eder.

Aygıt Sürücüsü (Device Driver)

Device driver, belirli bir denetçi (controller) ile konuşan yazılımdır. Sürücü genellikle kernel modunda çalışır. Üç farklı yükleme yöntemi vardır:

Yöntem	Açıklama	Örnek
Kernel Yeniden Derleme	Sürücü ile kernel yeniden derlenir, sistem açılır	Eski UNIX sistemleri
Boot Zamanında Yükleme	OS açılışta sürücü dosyasını yükler	Eski Windows sürümleri
Dinamik Yükleme (Hot-plug)	Sistem çalışırken sürücü yüklenir/kaldırılır	USB, Thunderbolt (modern)

1.3.5 Bus Mimarisi

Modern x86 sisteminde birden fazla bus bulunur. Her biri farklı hız ve işleve sahiptir:

Bus	Bant Genişliği (tipik)	Kullanım Amacı
PCIe (Peripheral Component Interconnect Express)	16 GB/s (x16 Gen 4)	GPU, NVMe SSD, Ağ kartları
DDR4/DDR5 Bellek Bus'u	50-100 GB/s	CPU ↔ RAM
DMI (Direct Media Interface)	~4 GB/s	CPU ↔ Platform Denetçisi (PCH)
SATA	600 MB/s	HDD ve SATA SSD'ler
USB 3.2 / USB 4	2-5 GB/s	Dış çevre birimleri
Gigabit Ethernet	125 MB/s	Ağ iletişimi

1.3.6 Bilgisayarın Açılışı (Booting the Computer)

Bilgisayar açıldığında BIOS/UEFI firmware ROM'dan çalışır, donanımı başlatır (POST - Power On Self Test), ardından boot aygıtını bulup OS'in boot yükleyicisini (bootloader) çalıştırır. Bootloader işletim sistemi çekirdeğini (kernel) belleğe yükler ve kontrolü ona devreder.

```
BIOS/UEFI ROM'dan başlar
→ POST (Power On Self Test): donanım kontrolü
→ Boot aygıtı (disk/USB/ağ) belirlenir
→ MBR veya GPT boot sektörü okunur
→ Bootloader (GRUB / Windows Boot Manager) yüklenir
→ Bootloader → Kernel belleğe yüklenir
→ Kernel: init/systemd başlatılır
→ Kullanıcı arayüzü (login screen) gösterilir
```

1.4 İşletim Sistemi Hayvanat Bahçesi (The Operating System Zoo)

İşletim sistemleri tek tip değildir; farklı kullanım senaryoları için farklı tasarım kararları alınmıştır. Kitapta dokuz farklı OS kategorisi tanımlanmaktadır:

Kategori	Hedef Ortam	Örnekler	Belirleyici Özellik
Mainframe OS	Büyük kurumsal veri merkezleri	IBM z/OS	I/O ağırlıklı; yüksek güvenilirlik; toplu iş (batch)
Sunucu OS (Server OS)	Web/dosya/veritabanı sunucuları	Linux, FreeBSD, Windows Server	Çok kullanıcı; ağ hizmetleri; ölçeklenebilirlik
Kişisel Bilgisayar OS (PC OS)	Masaüstü & dizüstü	Windows 11, macOS, Ubuntu	Tek kullanıcı; GUI; geniş uygulama ekosistemi
Akıllı Telefon OS (Smartphone OS)	Mobil aygıtlar	Android (Linux), iOS (BSD)	Pil yönetimi; dokunmatik; sensörler
IoT / Gömülü OS (Embedded OS)	Akıllı ev, endüstriyel kontrol	FreeRTOS, VxWorks, Embedded Linux	Küçük bellek; tek amaç; donanım bağımlı
Gerçek Zamanlı OS (Real-Time OS)	Uçak, ameliyat robotu, fabrika	VxWorks, QNX, RTEMS	Kesin zamanlama garantisi (hard/soft RT)
Akıllı Kart OS (Smart Card OS)	Kredi kartı, SIM kart	JavaCard	Çok kısıtlı kaynak; güvenlik odaklı

⚠ Real-Time OS: Hard vs Soft

Hard Real-Time: Deadline (son teslim süresi) asla kaçırılmaz; gecikme felakete yol açar (uçak kontrol sistemi, kalp pili). Soft Real-Time: Deadline kaçırılabilir ama kalite düşer (video oynatma, çevrimiçi oyun).

1.5 İşletim Sistemi Kavramları (Operating System Concepts)

1.5.1 Süreçler (Processes)

Process (süreç), çalışmakta olan bir programdır. Her process aşağıdakilere sahiptir:

Bileşen	Açıklama
Adres Uzayı (Address Space)	Programın kod, veri ve yığın (stack) alanlarını içeren sanal bellek bölgesi
Program Sayacı (PC)	Sıradaki çalışacak komutun adresi
Yazmaçlar (Registers)	CPU'nun mevcut durumu — context switch sırasında kaydedilir
Process Control Block (PCB) / Process Table Entry	OS'in süreci yönetmek için tuttuğu tüm bilgi (durum, PID, ebeveyn, açık dosyalar...)
Açık Dosyalar	Sürecin kullandığı dosya tanımlayıcıları (file descriptors)

Süreç Durumu (State)	Açıklama
Running (Çalışıyor)	CPU'da aktif olarak yürütülüyor
Ready (Hazır)	Çalışmak için bekliyor; CPU müsait değil
Blocked (Engellenmiş)	Bir I/O veya olayı bekliyor; CPU'ya ihtiyacı yok

```
/* UNIX'te yeni process oluşturma */
pid_t pid = fork();           // Çocuk process yarat
if (pid == 0) {
    execve('/bin/ls', ...);   // Çocukta yeni program çalıştır
} else {
    waitpid(pid, ...);        // Ebeveyn çocuğu bekler
}
```

1.5.2 Adres Uzayları (Address Spaces)

Her process birbirinden bağımsız bir sanal adres uzayına sahiptir. Fiziksel RAM üzerindeki gerçek adresler MMU (Memory Management Unit) aracılığıyla süreç başına dönüştürülür. Bu sayede süreçler birbirinin belleğine erişemez.

Sanal Bellek Bölgesi	İçerik
Stack (Yığın)	Yerel değişkenler, fonksiyon parametreleri, dönüş adresleri
Heap (Öbek)	Dinamik bellek (malloc/new ile ayrılan)
BSS / Veri Segmenti	Başlangıç değeri olan/olmayan global değişkenler
Text (Kod) Segmenti	Çalıştırılabilir program kodu (genellikle salt okunur)

1.5.3 Dosyalar (Files)

Dosya sistemi, kalıcı veri depolamanın temel soyutlamasıdır. UNIX'te 'her şey bir dosyadır' (everything is a file) felsefesi hâkimdir: normal dosyalar, dizinler, aygıt dosyaları, soketler, pipe'lar hepsi aynı arayüzle kullanılır.

Kavram	Açıklama	Örnek
Dosya (File)	Adlandırılmış kalıcı veri birimi	rapor.pdf, /etc/passwd
Dizin (Directory)	Dosyaları gruplayan hiyerarşik yapı	/home/ali/belgeler/
Yol (Path)	Mutlak: kökten (/), Göreli: mevcut dizinden	/usr/bin/python3 vs ./prog
Mount (Bağlama)	Yeni dosya sistemini mevcut ağaca ekleme	mount /dev/sdb1 /mnt/usb
File Descriptor (fd)	Açık dosyayı temsil eden tamsayı	0=stdin, 1=stdout, 2=stderr
Pipe (Boru)	İki process arasında tek yönlü iletişim kanalı	ls grep .txt

```
/* Temel dosya işlemleri (POSIX) */
int fd = open('dosya.txt', O_RDONLY);
char buf[1024];
ssize_t n = read(fd, buf, sizeof(buf));
write(1, buf, n); // stdout'a yaz
close(fd);
```

1.5.5 & 1.5.6 Koruma ve Shell

Kavram	Açıklama
Protection (Koruma)	UNIX'te her dosyanın sahibi, grubu ve izin bitleri vardır (rwx). Bu sayede hangi kullanıcının neye erişebileceği kontrol altındadır.
UID / GID	User ID / Group ID: her kullanıcı ve gruba atanan sayısal kimlik. root (UID=0) tüm kaynaklara erişebilir.
Shell	Metin tabanlı kullanıcı arayüzü; OS sistem çağrılarını kullanarak komutları çalıştıran program (bash, zsh, PowerShell vb.)
GUI (Graphical User Interface)	İkon ve pencere tabanlı arayüz (Gnome, KDE, macOS Aqua, Windows Shell). OS'in parçası değil; OS API'sini kullanan uygulama.

1.6 Sistem Çağrıları (System Calls)

Kullanıcı modu programları donanıma doğrudan erişemez; bunun yerine kernel'den hizmet istemek için sistem çağrısı (system call) yapar. System call bir özel procedure call türüdür; çalıştırıldığında CPU user mode'dan kernel mode'a geçer, OS istenilen servisi yerine getirir ve sonucu döndürür.

System Call Mekanizması

```
/* Kullanıcı programı system call yapıyor (POSIX C) */
ssize_t n = read(fd, buffer, nbytes);

/* Arka planda olan (basitleştirilmiş): */
// 1. Parametreler yazmaçlara / yığına yazılır
// 2. SYSCALL / INT 0x80 talimatı çalıştırılır → user→kernel mode
// 3. Kernel: system call numarasını kontrol eder
// 4. Kernel: ilgili handler'ı (sys_read) çalıştırır
// 5. Sonuç yazmaçlara yazılır → kernel→user mode
// 6. Kullanıcı program kaldığı yerden devam eder
```

i Trap vs Interrupt

Trap vs Interrupt farkı: Trap (tuzak) yazılım kaynaklıdır — program kasıtlı olarak tetikler (system call, sıfıra bölme). Interrupt (kesme) donanım kaynaklıdır — herhangi bir anda, programdan bağımsız olarak gelir (ağ paketi, disk tamamlandı, klavye tuşu).

Önemli POSIX Sistem Çağrıları

Kategori	Çağrı	Açıklama
Süreç Yönetimi	fork()	Mevcut süreci kopyalayarak çocuk süreç oluştur
Süreç Yönetimi	execve(path, argv, envp)	Süreci yeni bir programla değiştir
Süreç Yönetimi	exit(status)	Süreci sonlandır ve çıkış kodunu bildir
Süreç Yönetimi	waitpid(pid, &status;, opts)	Çocuk sürecin bitmesini bekle
Süreç Yönetimi	getpid() / getppid()	Kendi / ebeveyn PID'ini döndür
Dosya Yönetimi	open(path, flags, mode)	Dosyayı açar, fd döndürür
Dosya Yönetimi	read(fd, buf, n)	fd'den en fazla n byte oku
Dosya Yönetimi	write(fd, buf, n)	fd'ye n byte yaz
Dosya Yönetimi	close(fd)	Dosyayı kapat, fd'yi serbest bırak
Dosya Yönetimi	lseek(fd, offset, whence)	Dosya işaretçisini konumla
Dizin Yönetimi	mkdir(path, mode)	Yeni dizin oluştur
Dizin Yönetimi	rmdir(path)	Boş dizini sil
Dizin Yönetimi	link(old, new)	Hard link oluştur
Dizin Yönetimi	unlink(path)	Dosyayı sil (link sayısı düşer)
Dizin Yönetimi	mount(dev, dir, fstype, ...)	Dosya sistemini bağla
Diğer	kill(pid, signal)	Sürece sinyal gönder
Diğer	alarm(secs)	Belirtilen süre sonra SIGALRM gönder
Diğer	pipe(fd[2])	Anonim pipe yarat
Diğer	ioctl(fd, request, ...)	Aygıtı özel kontrol komutu gönder

POSIX vs Win32 API Karşılaştırması

İşlem	POSIX (UNIX/Linux)	Win32 (Windows)
Process Oluşturma	fork() + execve()	CreateProcess()
Dosya Açma	open()	CreateFile()
Dosya Okuma	read()	ReadFile()
Dosya Yazma	write()	WriteFile()
Bellek Ayırma	mmap() / brk()	VirtualAlloc()
Thread Oluşturma	pthread_create()	CreateThread()
Dinamik Kütüphane	dlopen() / dlsym()	LoadLibrary() / GetProcAddress()

1.7 İşletim Sistemi Yapısı (Operating System Structure)

Büyük ve karmaşık sistemler olan işletim sistemleri çeşitli mimari modellerle tasarlanmıştır. Her modelin güçlü ve zayıf yönleri vardır.

Mimari	Açıklama	Avantajlar	Dezavantajlar	Örnekler
Monolitik (Monolithic)	Tüm OS tek büyük çekirdek programı; her bileşen birbirine doğrudan çağrı yapabilir	Hızlı; prosedür çağırısı maliyeti düşük; basit iletişim	Hata bir yerde çöker tümü çöker; bakımı zor; test edilmesi güç	Linux, BSD kernels (loadable modules ile)
Katmanlı (Layered)	Her katman sadece altındaki katmanın servislerini kullanır; soyutlama hiyerarşisi	Modülerlik; katman katman doğrulama mümkün	Katman sınırları geçişi performans kaybı yaratır; katman belirleme zor	THE OS (Dijkstra, 1968) MULTICS tasarım hedefi
Mikroçekirdek (Microkernel)	Çekirdekte yalnızca minimum işlevsellik: IPC, temel bellek, çizelgeleme. Geri kalanı kullanıcı alanında	Güvenilirlik; sürücü çökmesi OS'i çökertmez; taşınabilirlik	Mesaj geçişi (IPC) maliyeti; monolitiğe göre yavaş	MINIX 3, QNX macOS çekirdeği (Mach)
İstemci-Sunucu (Client-Server)	Servisler ayrı süreçlerde çalışır; Mikroçekirdek ilkesinin bütün ağa uzatılması	Dağıtık sistemlere uygun; hata izolasyonu	Ağ gecikmesi; karmaşık IPC	Ağ dosya sistemleri (NFS)
Sanal Makine (Virtual Machine / Hypervisor)	Hypervisor gerçek donanımı virtualize eder; her VM kendi OS'ini tam çalıştırır	Tam izolasyon; farklı OS'ler aynı anda; güvenlik sandbox	Performans yükü (overhead); karmaşıklık	VMware ESXi (Type 1) VirtualBox (Type 2) Hyper-V, KVM
Exokernel / Unikernel	Exo: Donanım kaynaklarını uygulamaya doğrudan tahsis et Uni: Tek uygulamaya özel OS	Maksimum performans; minimum katman	Karmaşık programlama; güvenlik zor	MirageOS (Unikernel) Exokernel (MIT araştırması)

Sanal Makineler — Tip 1 vs Tip 2 Hypervisor

Özellik	Tip 1 (Bare-metal Hypervisor)	Tip 2 (Hosted Hypervisor)
Konum	Doğrudan donanım üzerinde çalışır	Konak OS üzerinde çalışır
Performans	Yüksek (düşük overhead)	Daha düşük (çift katman)
Kullanım	Veri merkezi, kurumsal	Kişisel kullanım, geliştirme
Örnekler	VMware ESXi, Hyper-V, Xen, KVM	VirtualBox, VMware Workstation
Güvenlik	Daha güvenli izolasyon	Konak OS zafiyetlerinden etkilenir

i JVM: Farklı Bir Sanal Makine

JVM (Java Virtual Machine) da bir sanal makine örneğidir — ancak donanım değil uygulama düzeyinde çalışır. Bytecode'u yorumlayarak platform bağımsızlığı sağlar. Bu, Type 2 hypervisor'a benzer ancak çok daha üst katmanda bir soyutlamadır.

Bölüm Özeti — Temel Kavramlar Sözlüğü

Terim (İngilizce)	Türkçe Karşılığı	Kısa Açıklama
Operating System (OS)	İşletim Sistemi	Donanım ve uygulama arasındaki yazılım katmanı
Kernel Mode	Çekirdek Modu	CPU'nun tüm komutlara erişebildiği mod
User Mode	Kullanıcı Modu	Kısıtlı komut seti; I/O yasak
System Call	Sistem Çağrısı	User→Kernel geçişini sağlayan mekanizma
Process	Süreç	Çalışan program + durumu + adres uzayı
Address Space	Adres Uzayı	Bir sürece ait sanal bellek bölgesi
Interrupt	Kesme	Donanım kaynaklı asenkron sinyal
Trap	Tuzak	Yazılım kaynaklı senkron istisna (hata veya syscall)
DMA	Doğrudan Bellek Erişimi	CPU müdahalesi olmadan bellek↔aygıt transferi
Cache	Önbellek	Sık kullanılan verinin hızlı bellekte tutulması
MMU	Bellek Yönetim Birimi	Sanal→fiziksel adres dönüşümü yapan donanım
Multiprogramming	Çok Programlılık	Birden fazla programın bellekte aynı anda bulunması
Timesharing	Zaman Paylaşımı	CPU'nun çok kullanıcı/program arasında hızla paylaşımı
Spooling	Eş Zamanlı Çevre Birimi İşlemi	I/O çıktısını önce tampona yazma tekniği
Hypervisor	Sanal Makine Monitörü	Donanımı sanallaştıran yazılım katmanı
Device Driver	Aygıt Sürücüsü	Belirli bir controller ile konuşan kernel yazılımı
Shell	Kabuk	Kullanıcı komutlarını OS'e ileten arayüz programı
File Descriptor	Dosya Tanımlayıcı	Açık dosyayı temsil eden tamsayı (fd)
Pipe	Boru	Süreçler arası tek yönlü veri kanalı
Boot Loader	Önyükleyici	Kernel'i belleğe yükleyen yazılım (GRUB vb.)

Açık Uçlu Quiz Soruları ve Cevapları

i Nasıl Çalışmalısınız

Aşağıdaki sorular sınav formatındadır. Her soruyu kendi başınıza yanıtlamaya çalışın, ardından cevap bölümünü kontrol edin.

S1. İşletim sisteminin iki temel işlevini açıklayın ve her biri için somut bir örnek verin.

Cevap:

İşletim sisteminin iki temel işlevi şunlardır:

(1) Genişletilmiş Makine (Extended Machine): Karmaşık donanım ayrıntılarını gizleyerek uygulama programlarına soyut ve kullanışlı bir arayüz sunar. Örnek: Bir C programcısı `write(fd, buf, n)` çağırısı yapar; altta disk controller'ına gönderilen yüzlerce komut tamamen gizlidir.

(2) Kaynak Yöneticisi (Resource Manager): CPU, bellek, disk gibi kaynakları birden fazla program arasında adil ve verimli biçimde paylaştırır. Örnek: 3 program aynı anda yazıcıya çıktı göndermek isterse OS spooling ile sıraya koyar, karışıklığı önler.

S2. Kernel mode ile user mode arasındaki farkı açıklayın. Bu ayrım neden gereklidir?

Cevap:

Kernel mode: CPU'nun tüm komut setine, tüm bellek adreslerine ve tüm donanım kaynaklarına erişebildiği özel ayrıcalıklı çalışma modudur. PSW'deki tek bir bit bu modu kontrol eder.

User mode: Yalnızca belirli (zararsız) komutların çalıştırılabildiği kısıtlı moddur. I/O komutları, bellek koruma ayarları ve PSW mod biti değiştirme user mode'da yasaktır.

Neden gerekli? Bir kullanıcı programının kasıtlı veya kazara sistem kaynaklarına zarar vermesi, diğer kullanıcıların verilerini okuması veya OS'i çöktürmesi engellenir. Güvenlik ve kararlılık bu ayrımla sağlanır.

S3. DMA (Direct Memory Access) nedir? Neden interrupt-driven I/O'ya göre tercih edilir?

Cevap:

DMA, bellekten I/O aygıtına veya tersi yönde veri transferini CPU müdahalesi olmaksızın gerçekleştiren özel bir donanım birimidir.

Interrupt-driven I/O: Her byte/word transferinde CPU kısa da olsa işe karışır.

DMA ile: CPU yalnızca transferi başlatır (başlangıç/bitiş adresi, byte sayısı) ve serbest kalır. DMA transfer bittiğinde tek bir interrupt gönderir. Büyük blok transferlerinde (örneğin 10 MB dosya kopyalama) CPU kullanımı dramatik biçimde azalır ve diğer işler için CPU

mevcuttur.

S4. Monolitik kernel ile microkernel mimarisini karşılaştırın. Her birinin avantaj ve dezavantajlarını verin.**Cevap:**

Monolitik Kernel: OS'in tüm bileşenleri (dosya sistemi, bellek yönetimi, sürücüler, ağ) tek bir büyük çekirdek programında çalışır. Avantaj: Prosedür çağırısı ucuzdur, yüksek performans. Dezavantaj: Bir hata (buggy driver) tüm sistemi çökertir; bakımı ve test edilmesi zordur. Örnek: Linux (loadable modules ile).

Microkernel: Çekirdekte yalnızca minimum işlevler (IPC, temel bellek, zamanlayıcı) kalır; sürücüler, dosya sistemi kullanıcı alanında ayrı süreçler olarak çalışır. Avantaj: Bir sürücü çökerse OS ayakta kalır; güvenilirlik ve taşınabilirlik yüksek. Dezavantaj: Mesaj geçişi (IPC) maliyeti nedeniyle daha yavaş. Örnek: MINIX 3, QNX, macOS (Mach çekirdeği).

S5. Process state'leri (süreç durumları) açıklayın. Running, Ready ve Blocked arasındaki geçişleri tetikleyen olayları belirtin.**Cevap:**

Running (Çalışıyor): Süreç CPU üzerinde aktif olarak yürütülmektedir. Aynı anda yalnızca çekirdek sayısı kadar süreç bu durumda olabilir.

Ready (Hazır): Süreç çalışmaya hazır ama CPU başka bir süreci çalıştırıyor. CPU müsait olunca scheduler (zamanlayıcı) bu süreçlerden birini seçer.

Blocked (Engellenmiş): Süreç bir olayı bekliyor (I/O tamamlanması, kilit serbest bırakılması, timer vb.) CPU'ya ihtiyacı yok.

Geçişler: Running→Ready: zamanlayıcı preemption (zaman dilimi doldu). Running→Blocked: I/O isteği, bekleme. Blocked→Ready: I/O tamamlandı, olay gerçekleşti. Ready→Running: scheduler süreci seçti.

S6. Sanal bellek (virtual memory) nedir? MMU bu süreçte ne rol oynar?**Cevap:**

Sanal bellek, her sürecin fiziksel RAM'den çok daha büyük bir adres uzayına sahipmiş gibi davranmasını sağlayan mekanizmadır. Programın aktif olarak kullanılan kısımları RAM'de tutulur; geri kalanı disk/SSD'ye (swap alanına) yazılır.

MMU (Memory Management Unit) CPU içindeki donanım birimidir. Her bellek erişiminde sürecin ürettiği sanal adresi fiziksel RAM adresine dönüştürür. Bu dönüşüm sayfa tabloları (page tables) aracılığıyla yapılır. Eğer istenen sayfa RAM'de yoksa page fault oluşur, OS ilgili sayfayı diskten yükler.

S7. Spooling kavramını açıklayın ve modern sistemlerde hâlâ kullanılıp kullanılmadığını tartışın.**Cevap:**

Spooling (Simultaneous Peripheral Operation On Line): Birden fazla işin çıktısını doğrudan yavaş bir çevre birimine (yazıcı, delikli kart okuyucu) göndermek yerine önce hızlı bir ara belleğe (disk/SSD) yazma tekniğidir. Orijinal olarak 2. nesil batch sistemlerde ortaya çıkmıştır.

Modern sistemlerde spooling yaygın biçimde kullanılmaktadır. Yazıcı sırası (print spooler — Linux'ta CUPS, Windows'ta Print Spooler servisi) bunun en iyi örneğidir. E-posta kuyruğu, iş kuyruğu (job queue) sistemleri de spooling mantığını paylaşır. Dolayısıyla cevap: Evet, modern bilgisayarlarda spooling hâlâ standar bir özelliktir.

Kitabın Bölüm Sonu Soruları ve Tam Çözümleri

❏ Kaynak: Kitap Sonu Problemleri

Aşağıdaki sorular Tanenbaum & Bos, Modern Operating Systems 5. Baskı, Bölüm 1 sonu problemlerinden seçilmiştir. Her soru için tam çözüm verilmiştir.

Problem 1.

İşletim sisteminin iki ana işlevi nedir? (What are the two main functions of an operating system?)

Çözüm:

Cevap:

1. Genişletilmiş Makine (Extended Machine / Top-Down View): Donanımın karmaşık ve çirkin arayüzünü, uygulama programlarına güzel, soyut ve kullanışlı bir arayüze dönüştürmek. Dosya, süreç, soket gibi soyutlamalar bu işlevin ürünüdür.
2. Kaynak Yöneticisi (Resource Manager / Bottom-Up View): CPU, bellek, disk, ağ kartı gibi donanım kaynaklarını birden fazla program ve kullanıcı arasında adil, verimli ve güvenli biçimde yönetmek; çakışan talepleri tahkim etmek.

Problem 2.

Multiprogramming (çok programlılık) nedir?

Çözüm:

Cevap:

Multiprogramming, birden fazla programın aynı anda bellekte bulunduğu ve CPU'yu paylaştığı tekniktir. Bir program I/O için beklerken (CPU'ya ihtiyaç duymadığı süreçte) CPU başka bir programa tahsis edilir. Bu sayede CPU boş kalmaz, sistem verimi (throughput) artar. 3. nesil OS'lerle (1965–1980) ana akım haline gelmiştir. Örnek: Belleğe 3 iş yüklenmiş; iş 1 I/O beklerken CPU iş 2'yi, o da beklerken iş 3'ü çalıştırır.

Problem 4.

Önbellek belleği kullanmak için ana bellek cache line'lara bölünür (genellikle 32 veya 64 byte). Tek byte veya word yerine tüm cache line'ın bir defada önbelleklenmesinin avantajı nedir? (What is the advantage of caching an entire line instead of a single byte or word?)

Çözüm:

Cevap:

Yerellik ilkesi (locality of reference) sayesinde: Programlar hem zamansal yerellik (temporal locality — kısa süre önce erişilen veriye tekrar erişilir) hem de mekansal yerellik (spatial locality — bir veriye erişildiğinde yakınındaki verilere de yakın zamanda erişilir) gösterir.

Tüm cache line yüklenince, bir sonraki erişim büyük ihtimalle aynı veya komşu cache line'da olacağından cache hit oranı dramatik biçimde artar. Tek byte yüklemek bir dizi erişim (for döngüsü gibi) sürekli cache miss üretirdi; 64 byte'lık blok bunu engeller.

Ayrıca bellek veri yolunun burst transfer kapasitesinden faydalanılır; her seferinde 1 byte yerine 64 byte'ı bir seferde taşımak çok daha verimlidir.

Problem 5.

Spooling nedir? Gelecekte gelişmiş kişisel bilgisayarların standart bir özelliği olacak mı?

Çözüm:

Cevap:

Spooling (Simultaneous Peripheral Operation On Line): Yavaş bir çevre birimine (yazıcı) doğrudan gönderilecek çıktının önce hızlı bir ara belleğe (disk) yazılması tekniğidir. CPU, programın disk yazımını beklemeden çalışmaya devam eder; yazıcı da kendi hızında disk tamponundan okur.

Gelecek için: Evet. Modern kişisel bilgisayarlarda spooling standart bir özelliktir (CUPS yazıcı kuyruğu, Windows Print Spooler). Disk depolama ucuzladıkça ve çevre birimleri artık ağ üzerinden paylaşıldıkça bu tekniğin önemi daha da artmaktadır.

Problem 8.

I/O aygıtlarına erişim komutları neden ayrıcalıklı komut (privileged instruction) — yani yalnızca kernel modunda çalıştırılabilir? Bir neden verin.

Çözüm:

Cevap:

Güvenlik ve izolasyon: Eğer her kullanıcı programı I/O aygıtlarına doğrudan erişebilseydi şunlar olabilirdi: (a) Kötü niyetli bir program başka kullanıcıların disk bloklarını okuyabilir, hassas verileri ele geçirebilirdi. (b) Bir program yazıcıya sonsuz çıktı göndererek aygıtı ele geçirebilirdi. (c) Ağ kartına doğrudan erişimle sahte paketler gönderilebilirdi. OS, I/O komutlarını ayrıcalıklı kılarak tüm I/O erişimini denetler, kimin hangi aygıtı erişeceğini izin sistemiyle kontrol eder.

Problem 11.

Kernel modu ile kullanıcı modu arasındaki fark nedir? İki ayrı mod bulundurmak işletim sistemi tasarımına nasıl katkıda bulunur? (What is the difference between kernel and user mode? Explain how having two distinct modes aids in designing an operating system.)

Çözüm:

Cevap:

Kernel mode: CPU PSW'deki mod biti = 0 (ayrıcalıklı). Tüm komutlar, tüm bellek adresleri ve tüm donanım kaynakları erişilebilir. İşletim sistemi çekirdeği bu modda çalışır.

User mode: Mod biti = 1 (kısıtlı). Yalnızca zararsız komutlar; I/O, bellek koruma, mod biti değiştirme yasak. Uygulama programları bu modda çalışır.

OS tasarımına katkısı:

- Korunma (Protection): Kullanıcı programlar OS kodunu ve diğer süreçlerin belleğini değiştiremez. Kazara veya kasıtlı sistem hasarı engellenir.
- Kararlılık (Stability): Bir uygulama çökse bile OS ayakta kalır; sadece o süreç sonlanır.
- Güvenlik (Security): Ayrıcalıklı işlemler (I/O, bellek yönetimi) yalnızca OS tarafından yapılabilir; kullanıcı programlar system call aracılığıyla hizmet alır, böylece OS tüm talepleri denetleyebilir.

Problem 15.

Bir bilgisayarda 4 aşamalı pipeline bulunmaktadır. Her aşama aynı sürede çalışmakta olup 1 ns sürmektedir. Bu makine saniyede kaç komut çalıştırabilir? (A computer has a pipeline with four stages, each taking 1 nsec. How many instructions per second can this machine execute?)

Çözüm:

Cevap:

Pipeline'ın fikri: Pipeline dolduğunda her clock cycle'da bir komut tamamlanır. Pipeline derinliği (aşama sayısı) komut başına süreyi değil, tek bir komutun toplam gecikmesini (latency) etkiler; ancak verim (throughput) pipeline boyunca 1 komut/cycle olarak kalır.

Her aşama 1 ns → Clock periyodu $T = 1 \text{ ns} \rightarrow f = 1/T = 1 \text{ GHz}$

Dolayısıyla: $1 \text{ komut/ns} = 10^9 \text{ komut/saniye} = 1 \text{ GIPS (Giga Instructions Per Second)}$

Dikkat: İlk 4 aşamada pipeline dolana kadar (4 ns) tam verim elde edilemez; ancak uzun süren programlarda bu başlangıç gecikmesi ihmal edilebilir.

Problem 16.

Bir bilgisayar sisteminde cache bellek, ana bellek (RAM) ve disk bulunmaktadır. Cache erişimi 2 ns, RAM erişimi 10 ns, disk erişimi 10 ms'dir. Cache hit oranı %95, cache miss sonrası RAM hit oranı %99 ise ortalama kelime erişim süresi nedir? (What is the average time to access a word?)

Çözüm:

Cevap:

Olasılıklar:

- Cache hit: %95 → 2 ns
- Cache miss + RAM hit: $(1-0.95) \times 0.99 = 0.0495 \rightarrow 10 \text{ ns}$
- Cache miss + RAM miss (disk'e git): $(1-0.95) \times (1-0.99) = 0.0005 \rightarrow 10 \text{ ms} = 10,000,000 \text{ ns}$

Ortalama erişim süresi:

$$= 0.95 \times 2 + 0.0495 \times 10 + 0.0005 \times 10,000,000$$

$$= 1.9 + 0.495 + 5000$$

$$= 5002.395 \text{ ns} \approx 5002 \text{ ns} \approx 5 \mu\text{s}$$

Yorum: Disk erişiminin son derece nadir (%0.05) gerçekleşmesine rağmen ortalama süreyi tamamen domine ettiği görülmektedir. Bu, page fault'ların performans üzerindeki yıkıcı etkisini açıkça göstermektedir ve OS'in disk erişimini minimize etmesinin ne kadar kritik olduğunu kanıtlar.

Problem 18.

Trap ile interrupt arasındaki temel fark nedir? (What is the key difference between a trap and an interrupt?)

Çözüm:

Cevap:

Interrupt (Donanım Kesmesi): Donanım tarafından, çalışmakta olan programdan bağımsız olarak herhangi bir anda üretilen asenkron sinyaldir. Program bir komutun ortasında bile kesilebilir. Örnekler: disk transferi tamamlandı, ağ paketi geldi, klavyeye basıldı, timer süresi doldu.

Trap (Yazılım Kesmesi / İstisna): Program tarafından kasıtlı olarak (system call) veya bir hata sonucu (sıfıra bölme, geçersiz bellek erişimi, sayfa hatası) tetiklenen senkron olaydır. Belirli bir komutun çalıştırılması sonucu üretilir.

Özet: Interrupt asenkron ve dışsal; Trap senkron ve içseldir. İkisi de CPU'yu kernel moduna geçirir ve ilgili handler çalışır.

Problem 23.

Aşağıdaki kaynakların hangilerinin paylaşılmasında zaman çaklma (time), uzay çaklma (space) ya da her ikisi kullanılabilir: CPU, bellek, SSD/disk, ağ kartı, yazıcı, klavye, ekran?

Çözüm:

Cevap:

- CPU: Zaman çaklma (zamanlayıcı farklı süreçleri CPU'ya sırayla alır)
- Bellek (RAM): Uzay çaklma (farklı süreçler belleğin farklı bölgelerini kullanır)
- SSD/Disk: Uzay çaklma (farklı kullanıcıların dosyaları farklı sektörlerde); kısmen zaman çaklma (erişim sıralaması zamanlayıcıyla yapılır)
- Ağ Kartı: Zaman çaklma (paketler sırayla gönderilir/alınır)
- Yazıcı: Zaman çaklma (spooling ile iş sıraya alınır)
- Klavye: Zaman çaklma (hangi pencerenin giriş aldığı odak (focus) ile belirlenir)
- Ekran: Uzay çaklma (pencereler ekranın farklı bölgelerini kaplar); zaman çaklma (GPU render sıralaması)

Bölüm 1 — Giriş tamamlandı.

Modern İşletim Sistemleri · Tanenbaum & Bos · 5. Baskı

→ Sonraki Adım: Bölüm 2 — Süreçler ve Thread'ler

Bir sonraki bölüm (Bölüm 2) Süreçler ve Thread'leri işlemektedir. Bu bölümden devam edilecek kavramlar: process creation (fork/exec), süreç durumları, thread'ler, senkronizasyon (mutex, semaphore) ve zamanlayıcı (scheduler).